

SWE PROJECT: TXST Lynx

Part 1: Project Proposal

Group Members: Sachin, Misan, Adam B, Adam S, Daniel, Seth

The Vision

TXST Lynx is a web-based social networking platform designed specifically for the Texas State University community. The project aims to provide students with a centralized digital space for communication, community discovery, campus interaction, and social engagement. This social-networking platform will organize parts of the user experience around the structure and identity of Texas State University's colleges, acting as distinct communities within the larger network.

The project will combine conventional social-network functionality, like posting images, with a more visually distinctive interface. Most information-heavy portions of the application will use standard web interfaces, while limited animation and dynamic scrolling will enhance the visual experience of college-navigation, giving the application its own identity.

Goals

The primary goals of TXST Lynx are to:

- ❖ Create a social networking platform centered on Texas State University students and communities.
- ❖ Allow users to create profiles and participate in a shared campus social feed.
- ❖ Organize users and content around university colleges and other campus communities.
- ❖ Allow users to create and interact with posts through standard social features such as comments and reaction-icons.
- ❖ Provide users with ways to discover other students, groups, and campus communities [via search bars and interest preferences].
- ❖ Establish a maintainable separation between frontend, backend, and database layers.
- ❖ Create a visually distinctive experience through animation and dynamic page behavior without sacrificing usability.
- ❖ Develop a contained college-navigation experience that represents Texas State's colleges through polished interactive GUI.
- ❖ Produce a project architecture that can be divided effectively among a small software engineering team.
- ❖ Deliver a stable semester-scale prototype that demonstrates both technical engineering and thoughtful user experience.

Scope Boundary

❖ In Scope:

- User accounts and authentication
- University email address verification
- User profiles
- College affiliation
- Social feed
- Creating and viewing posts
- Comments & reactions
- Following / Connecting with other users
- College-specific communities or feeds
- Community discovery via basic search functionality
- Animated backgrounds and transitions
- Dynamic scrolling effects
- Interactive college banner/crest navigation GUI
- Backend REST API
- Relational database persistence
- Responsive web design where practical
- Small-scale machine-learning content moderation system

❖ Out of Scope:

- 3D GUI elements (unless we have time to expand scope)
- Native iOS/Android application
- Production-scale deployment infrastructure
- Complex recommendation algorithms
- Machine-learning content recommendations
- Advertising systems
- Payment processing
- Video streaming infrastructure
- Complex real-time communication infrastructure
- Full university administrative integration
- Production-scale analytics

List of Features

❖ User Accounts

- Users will be able to create an account and authenticate with the application.
- Account information includes:
 - Username
 - Display name
 - Email
 - Password/authentication credentials
 - College affiliation
 - Major or academic interests
 - Profile image
 - Biography

❖ Main Social Feed

- The application will contain a primary feed where users can browse recent or relevant campus posts.
- Users will be able to:
 - View posts
 - Create posts
 - Open individual posts
 - Comment and react to posts
 - Navigate to the author's profile
- The initial implementation should favor a simple chronological feed over a complicated recommendation algorithm.

❖ Posts

- Users will be able to publish content to the platform.
- The initial post model will support:
 - Images
 - Text
 - Author
 - Creation timestamp
 - College/community association
 - Reactions
 - Comments

❖ College Communities

- Texas State's colleges will serve as major organizational communities within the platform.
- Users will be able to:
 - Associate their account with a college
 - Browse a college-specific page
 - View posts associated with that college
 - Discover students associated with that college
 - Navigate between college communities
- This system provides a university-specific structure that differentiates TXST Lynx from a generic social networking application.

❖ Groups and Community Discovery

- Users will be able to discover or participate in smaller communities based on:
 - Student interests
 - Academic subjects
 - Organizations
 - Events
 - Hobbies
- The complexity of group creation and administration will depend on available development time.

❖ Search and Discovery

- The application should provide a basic method for discovering:
 - Users
 - Colleges
 - Communities or groups

❖ Animated User Interface

- TXST Lynx will have a stronger visual identity than a standard CRUD application.
- Potential interface features include:
 - Animated backgrounds
 - Smooth transitions
 - Scroll-based animations
 - Parallax effects
 - Dynamic page elements
 - Animated navigation
- GSAP may be used for advanced animation where appropriate

List of Technical Requirements

- ❖ **Front End** – communicates with the backend through HTTPS/JSON API requests.
 - Next.js
 - React
 - TypeScript
 - Tailwind CSS
 - ESLint
- ❖ **Backend** – responsible for API endpoints, application logic, request validation, and DB ops.
 - Python
 - FastAPI
 - uv by Astral (for Python dependency/env management)
- ❖ **Database**
 - PostgreSQL is planned as the primary relational database, containing entities such as:
 - Users
 - Profiles
 - Colleges
 - Posts
 - Comments
 - Reactions
 - Follows
 - Groups
 - Group Membership
 - Supabase will be used to provide hosted PostgreSQL and potentially authentication, storage, and real-time services.
- ❖ **Architecture**
 - Browser
 - Next.js / React / TypeScript
 - HTTP / JSON API
 - FastAPI / Python
 - SQLAlchemy
 - PostgreSQL
 - UI
 - Next.js / React: components and structure
 - Tailwind CSS: appearance and layout
 - GSAP: animation, transitions, scrolling effects, parallax, movement
 - Browser Rendering

Breakdown of Tasks

1) Project Foundation

- a) Create Git repository
- b) Configure Next.js frontend
- c) Configure FastAPI backend
- d) Establish frontend/backend communication
- e) Establish development workflow
- f) Document dependencies
- g) Configure .gitignore
- h) Establish team Git workflow

2) Database Foundation

- a) Configure PostgreSQL
- b) Connect FastAPI to PostgreSQL
- c) Configure SQLAlchemy (ORM plus database toolkit)
- d) Define initial database models
- e) Establish database initialization/migration strategy
- f) Test database reads and writes

3) Core Social Vertical Slice

Vertical Slice = Build one complete feature through every architectural layer.

Create Post → Next.js → FastAPI → PostgreSQL → Retrieve Post → Display in Feed

//This slice validates the application's full architecture before new features are introduced.

- a) Post database model
- b) Post API schema
- c) Create-post endpoint
- d) Retrieve-post endpoint
- e) Post creation interface
- f) Feed interface
- g) End-to-end testing

4) Accounts and Profiles

- a) Authentication
- b) User database model
- c) Profile model
- d) Registration/login interface
- e) Profile API
- f) Profile pages
- g) Profile editing
- h) College affiliation

5) Social Interaction

- a) Comments
- b) Reactions
- c) Follow relationships
- d) User discovery
- e) Feed integration
- f) Error handling and validation

6) College Community System

- a) College data model
- b) College pages
- c) College-specific feeds
- d) Associate users with colleges
- e) Associate posts with colleges
- f) College discovery/navigation

7) Groups and Additional Community Features

- a) Group model
- b) Group membership
- c) Group pages
- d) Group posts
- e) Community discovery
- f) Search functionality

8) Visual Enhancement

Once core functionality is stable:

- a) Establish application visual identity
- b) Animated backgrounds
- c) Page transitions
- d) Scroll effects
- e) GSAP integration
- f) Refine responsive layouts
- g) Improve loading/error states
- h) UI polish

9) Integration, Testing, & Final Presentation

- a) Full application integration
- b) Functional testing
- c) Database testing
- d) UI testing
- e) Browser testing
- f) Fix critical bugs
- g) Improve error handling
- h) Final Deployment

Timeline

- ❖ **August 31st - September 4th**
 - Architecture decisions locked in
 - Repository, Next.js and FastAPI scaffolds built and connected
- ❖ **September 7th – 11th**
 - PostgreSQL and SQLAlchemy configured, initial models, migration strategy
- ❖ **September 14th – 18th**
 - Post model and schema, create and retrieve endpoints
- ❖ **September 21st – 25th**
 - Post creation interface, feed interface, end-to-end test of full slice
- ❖ **September 28th – October 2nd**
 - Authentication, user model, registration and login, email verification
- ❖ **Oct 5th – 9th**
 - Profile model and API, profile pages, editing, college affiliation
- ❖ **Oct 12th – 16th**
 - Comments and reactions, persisted in database and displayed on feed
 - Visual identity work begins
- ❖ **Oct 19th – 23rd**
 - Follow relationships, user discovery, feed integration
 - Midterm demonstration
- ❖ **Oct 26th – 30th**
 - College model, college pages and feeds, user and post association
- ❖ **November 2nd – 6th**
 - Group model and membership, group pages and posts, search
- ❖ **November 9th – 13th**
 - Moderation classifier, remaining features finished or formally cut
- ❖ **November 16th – 20th**
 - Animation push: backgrounds, transitions, scroll effects, college crest navigation
- ❖ **November 23rd – 27th**
 - Thanksgiving light-work
 - Integration, documentation, final report
- ❖ **November 30th – December 4th**
 - Functional, database, UI and browser testing
 - Bug fixes, deployment
- ❖ **December 7th – 11th**
 - Final presentation, live demonstration, submission

Delegation of Responsibilities

Six Vertical Slices:

Each member owns a feature from database through API through Interface. All members want to learn full stack software development, so we won't have dedicated backend or database developers. Each member also holds shared-state cross-cutting concerns to carefully manage during collaboration.

❖ Sachin

- **Accounts & Authentication**
 - Cross-cutting: deployment, environments and CI

❖ Misan

- **Profiles & College Affiliation**
 - Cross-cutting: database schema, migration, seed data

❖ Adam B

- **Posts & Feed**
 - Cross-cutting: API contract and Open AI schema

❖ Adam S

- **Comments, Reactions & Follows**
 - Cross-cutting: testing and QA. Also owns the moderation classifier

❖ Daniel

- **College Communities, Groups & Search**
 - Cross-cutting: coordination, documentation, deliverables

❖ Seth

- **Visual Identity & Animation**
 - Cross-cutting: accessibility and responsive layout